תרגיל בית – OpenLibrary – Automation Engineer

מטרת התרגיל

- לממש תרחיש e2e על אתר OpenLibrary (openlibrary.org) הכולל חיפוש ספרים, סינון לפי שנת פרסום, הוספה ל־Reading List, ואימות רשימה.
- להציג ארכיטקטורה נקייה: **Page Object Model, OOP, Data-Driven**.

מסגרת זמן

4–5 שעות נטו ליישום + 20–30 דק' לקריאה/הדגמה.

דרישות כלליות

- **כלי אוטומציה:** Playwright | **שפה:** Python
- **דוחות:** Allure Reports / HTML + Screenshots
- פיתוח מונחה עצמים (OOP) במודל POM
- **Data-Driven:** קלטים מקובץ חיצוני (JSON / CSV / YAML)

---

1. פונקציית חיפוש עם תנאי שנה

```
async def search_books_by_title_under_year(query: str, max_year: int, limit: int = 5) -> list[str]:
```

- חיפוש לפי `query` , סינון לפי `max_year` , איסוף עד `limit` URLs.
- תמיכה ב־Pagination — מעבר לדף הבא אם יש פחות מ־5 תוצאות.
- החזרת 0 תוצאות תקינה אם אין התאמות.

```
urls = await search_books_by_title_under_year("Dune", 1980, 5)
```

2. הוספת ספרים ל־Reading List

```
async def add_books_to_reading_list(urls: list[str]) -> None:
```

- מעבר על כל URL, לחיצה על "Already Read" / "Want to Read" (רנדומלי).
- Screenshot + Log לכל ספר שנוסף.

3. אימות תוכן ה־Reading List

```
async def assert_reading_list_count(expected_count: int) -> None:
```

- פתיחת עמוד הרשימה, קריאת מספר הספרים, אימות מול `expected_count` .

- שמירת Screenshot/Trace.

4. מדידת ביצועים – Performance (45–60 דקות)

```
async def measure_page_performance(page, url: str, threshold_ms: int) -> dict:
```

- מדידת `load_time_ms` , `dom_content_loaded_ms` , `first_paint_ms` .
- חריגה מ־threshold = אזהרה בלוג (לא כישלון).
- הפקת `performance_report.json` בסיום.

נקודות מדידה: עמוד חיפוש (3000ms) | עמוד ספר (2500ms) | Reading List (2000ms)

תרגיל באגים – ניתוח סטטי (20 דקות)

הוראה: ללא שימוש בכלים/מחשב. כתוב ב־ `ReadMeAIBugs.md` לפחות 3 בעיות + הסבר + תיקון.

```
import asyncio
from playwright.async_api import async_playwright

BASE_URL = "https://openlibrary.org"

class BookSearchPage:
    def __init__(self, page):
        self.page = page
        self.search_input = "input[name='q']"
        self.search_button = "button[type='submit']"
        self.results = ".searchResultItem"

    async def search(self, query):
        await self.page.fill(self.search_input, query)
        await self.page.click(self.search_button)

class ReadingListPage:
    def __init__(self, page):
        self.page = page

    async def get_book_count(self):
        items = await self.page.query_selector_all(".listbook-item")
        return len(items)

async def search_books_by_title_under_year(page, query, max_year, limit=5):
    search_page = BookSearchPage(page)
    await search_page.search(query)
    collected = []
    while len(collected) < limit:
        results = await page.query_selector_all(".searchResultItem")
        for item in results:
            year_el = await item.query_selector(".bookEditions")
            if year_el:
                year_text = await year_el.inner_text()
                year = int(year_text.strip())
                if year <= max_year:
                    link = await item.query_selector("a")
                    href = await link.get_attribute("href")
                    collected.append(BASE_URL + href)
        next_btn = await page.query_selector(".next-page")
        if next_btn:
            await next_btn.click()
        else:
            break
    return collected

async def add_books_to_reading_list(page, urls):
    for url in urls:
        await page.goto(url)
        await page.click(".want-to-read-btn")
        await page.screenshot(path=f"screenshots/{url}.png")
```

```
async def assert_reading_list_count(page, expected_count):
    await page.goto(f"{BASE_URL}/account/books/want-to-read")
    reading_list = ReadingListPage(page)
    actual = reading_list.get_book_count()
    assert actual == expected_count, f"Expected {expected_count}, got {actual}"

async def main():
    async with async_playwright() as p:
        browser = await p.chromium.launch(headless=True)
        page = await browser.new_page()
        await page.goto(BASE_URL)
        urls = await search_books_by_title_under_year(page, "Dune", 1980, 5)
        await add_books_to_reading_list(page, urls)
        await assert_reading_list_count(page, len(urls))

asyncio.run(main())
```

מה להגיש

- קישור ל־GitHub עם גישה לריפו
- `README.md` — הרצה, ארכיטקטורה, מגבלות
- דוח ריצה (Allure / HTML / JUnit XML)
- `performance_report.json`
- `ReadMeAIBugs.md`

קריטריונים להערכה

תחום	משקל
ארכיטקטורה וניקיון קוד (POM, OOP, SRP, Utils)	40%
Robustness & Smart Locators (Pagination, פרסור שנה, סטטוסים)	30%
Performance (מדידות, threshold, דוח JSON)	15%
Data-Driven (קונפיגורציה, ENV, פרופילים)	10%
דוחות/תיעוד (README, Screenshots, Allure)	5%

— בהצלחה! 🚀 —